

Course Project: Deliverable 2

ERD, Relational Schema, and Integrity Constraints of a Hotel
Management System

CSI2132

Prof. Verena Kantere
University of Ottawa

Group 25	
Jamie Watson	300356613
Sabrina Seto	300359255
Alae Kabir	300357880

Section 1.a: Explanation of DBMS and Coding Languages Used

DBMS:

Our Database Management System is created using PostgreSQL, using the client tool pgAdmin 4. We decided to use PostgreSQL as it is a powerful, open source object relational database system. We were able to download, install, use and inspect the code for free. We were also familiar with the interface and coding language, due to the use of PostgreSQL in the course's laboratories.

pgAdmin 4 allowed us to easily execute queries and visualize our relations. The history of queries is also accessible for error tracing and re-using code.

Application Overview

The application uses a variety of technologies to create a web interface connected to a database. To begin, the application uses Apache Tomcat to handle the web interface and delegate HTTP requests to the required Java-based servlets. The backend of this application is written in Java and is divided into three key areas. First, the Servlets (found in the directory `/src/main/java/com/example/servlet` or [here](#)) each correspond to a specific task the application performs, such as creating a booking, and facilitate the transfer of requests to and from the frontend.

These servlets connect to the second key area, the Data Access Objects or DAO (found in the directory `/src/main/java/com/example/dao` or [here](#)). This section includes a `DBConnection` class that maintains a connection to the database running, which is then accessed by other classes in the directory to format and send SQL statements. This structure allows for the use of variables, increasing flexibility and handling in some SQL statements.

Because the SQL queries used throughout the application are heavily formatted within the Java code, raw SQL recreations of these queries have been prepared and located in the directory `/src/main/resources/database` or [here](#).

The third key area of the Java backend is the model directory, which contains Java class models representing key database entities, such as `Room.java` and `Customer.java`. Finally, the frontend exists in the directory `/src/main/webapp` or [here](#) and includes JSP files that send requests to the servlets. The interface is built using a combination of raw HTML and CSS, (also Bootstrap technologies for specific components), to create a functional and responsive web application interface.

Section 1.b: Installation Instructions

Prerequisites:

- Postgres must be installed. (Check with `psql --version`)
- Postgres user and password must be created
- Java installed (at least 11 or later)
- Maven installed
- Apache Tomcat installed (We use version 10)

Step 1 (Create and populate DB):

- Run the command: `psql -U postgres -c "CREATE DATABASE ehotel;"` which will create a database called "ehotel"
- Find SQL files Schema.sql and Data.sql which are in the directory `/src/main/resources/database` or can be found [here](#)
- Copy path of Schema.sql
- Create the Schemas for the database by running the following command:
`psql -U postgres -d ehotel -f "<path that was copied from previous step>"`

An example of running this command:

```
PS C:\Users\iamie\CS12132\E-Hotel-Project> psql -U postgres -d ehotel -f "C:\Users\iamie\CS12132\E-Hotel-Project\src\main\resources\database\schema.sql"
Password for user postgres:
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE TABLE
```

- Copy path of Data.sql
- Populate the data for the DB by running the following command:
`psql -U postgres -d ehotel -f "<path that was copied from previous step>"`

Step 2 (Java Configuration):

- Locate the DBConnection.java file which is in the directory /src/main/java/com/example/dao or can be found [here](#)
- Change the line in the password file to correspond to your set postgres password. I.E change `private static final String PASSWORD = "changeme";` to `private static final String PASSWORD = "<your password>";`
- Save the file
- Create the maven package by running the following command:
`mvn clean package`
- Deploy the Tomcat containing by running the following command:
`mvn cargo:run`
- Now the instance should be running on: `http://localhost:8080/`

Section 1.c: DDL Statement

Creating hotel_chain table	<pre>CREATE TABLE IF NOT EXISTS hotel_chain (hotel_chain_id integer primary key, street_number integer not null check (street_number > 0), street_name varchar(50) not null, city varchar(50) not null, province varchar(50) not null, zip varchar(6) not null CHECK (zip ~ '^[A-Za-z][0-9][A-Za-z][0-9][A-Za-z][0-9] \$'), num_of_hotels integer not null check (num_of_hotels >= 1), hotel_chain_name varchar(50) not null unique);</pre>
Create table hotel_chain_contact_phone	<pre>create table if not exists hotel_chain_contact_phone (hotel_chain_phone bigint primary key CHECK (hotel_chain_phone between</pre>

	<pre> 1000000000 and 9999999999), hotel_chain_id integer not null, foreign key (hotel_chain_id) references hotel_chain (hotel_chain_id) on delete cascade); </pre>
Create function limit_phone_per_chain	<pre> CREATE OR REPLACE FUNCTION limit_phone_per_chain() RETURNS TRIGGER AS \$\$ BEGIN IF (SELECT COUNT(*) FROM hotel_chain_contact_phone WHERE hotel_chain_id = NEW.hotel_chain_id) >= 5 THEN RAISE EXCEPTION 'A hotel chain cannot have more than 5 phone numbers'; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; </pre>
Create trigger phone_limit_trigger	<pre> CREATE TRIGGER phone_limit_trigger BEFORE INSERT ON hotel_chain_contact_phone FOR EACH ROW EXECUTE FUNCTION limit_phone_per_chain(); </pre>
Create table hotel_chain_contact_email	<pre> create table if not exists hotel_chain_contact_email (hotel_chain_email varchar(50) primary key, hotel_chain_id integer not null, foreign key (hotel_chain_id) references hotel_chain (hotel_chain_id) on delete cascade, </pre>

	<pre> CONSTRAINT chk_email_at CHECK (hotel_chain_email ~ '^[^\@]+\@[^\@]+\.[^\@]+\$')); </pre>
Create limit_email_per_chain	<pre> CREATE OR REPLACE FUNCTION limit_email_per_chain() RETURNS TRIGGER AS \$\$ BEGIN IF (SELECT COUNT(*) FROM hotel_chain_contact_email WHERE hotel_chain_id = NEW.hotel_chain_id) >= 5 THEN RAISE EXCEPTION 'A hotel chain cannot have more than 5 emails'; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; </pre>
Create trigger email_limit_trigger	<pre> CREATE TRIGGER email_limit_trigger BEFORE INSERT ON hotel_chain_contact_email FOR EACH ROW EXECUTE FUNCTION limit_email_per_chain(); </pre>
Create table hotel	<pre> create table if not exists hotel (hotel_id integer primary key, hotel_chain_id integer not null, street_number integer not null check (street_number > 0), street_name varchar(50) not null, city varchar(50) not null, province varchar(50) not null, zip varchar(6) not null CHECK (zip ~ </pre>

	<pre> '^[A-Za-z][0-9][A-Za-z][0-9][A-Za-z][0-9] \$'), hotel_name varchar(50) not null unique, num_of_rooms integer not null check (num_of_rooms >= 1), star_rating integer not null check (star_rating between 1 and 5), manager_ssn integer not null unique); </pre>
Create table hotel_contact_phone	<pre> create table if not exists hotel_contact_phone (hotel_phone bigint primary key CHECK (hotel_phone between 1000000000 and 9999999999), hotel_id integer not null, foreign key (hotel_id) references hotel (hotel_id) on delete cascade); </pre>
Create function limit_phone_per_hotel	<pre> CREATE OR REPLACE FUNCTION limit_phone_per_hotel() RETURNS TRIGGER AS \$\$ BEGIN IF (SELECT COUNT(*) FROM hotel_contact_phone WHERE hotel_id = NEW.hotel_id) >= 5 THEN RAISE EXCEPTION 'A hotel cannot have more than 5 phone numbers'; END IF; RETURN NEW; </pre>

	<pre> END; \$\$ LANGUAGE plpgsql; </pre>
Create trigger phone_limit_trigger_hotel	<pre> CREATE TRIGGER phone_limit_trigger_hotel BEFORE INSERT ON hotel_contact_phone FOR EACH ROW EXECUTE FUNCTION limit_phone_per_hotel(); </pre>
Create table hotel_contact_email	<pre> create table if not exists hotel_contact_email (hotel_email varchar(50) primary key, hotel_id integer not null, foreign key (hotel_id) references hotel (hotel_id) on delete cascade, CHECK (hotel_email ~ '^[^\@]+\@[^\@]+\.[^\@]+\$')); </pre>
Create function limit_email_per_hotel	<pre> CREATE OR REPLACE FUNCTION limit_email_per_hotel() RETURNS TRIGGER AS \$\$ BEGIN IF (SELECT COUNT(*) FROM hotel_contact_email WHERE hotel_id = NEW.hotel_id) >= 5 THEN RAISE EXCEPTION 'A hotel cannot have more than 5 emails'; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; </pre>
Create email_limit_trigger_hotel	<pre> CREATE TRIGGER email_limit_trigger_hotel BEFORE INSERT ON hotel_contact_email </pre>

	<pre>FOR EACH ROW EXECUTE FUNCTION limit_email_per_hotel();</pre>
Create table employee	<pre>CREATE TABLE if not exists employee (employee_ssn INTEGER PRIMARY KEY CHECK (employee_ssn between 100000000 and 999999999), hotel_id INTEGER NOT NULL, first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, street_number integer not null check (street_number > 0), street_name varchar(50) not null, city varchar(50) not null, province varchar(50) not null, zip varchar(6) not null CHECK (zip ~ '^[A-Za-z][0-9][A-Za-z][0-9][A-Za-z][0-9] \$'), FOREIGN KEY (hotel_id) REFERENCES hotel (hotel_id) ON DELETE CASCADE);</pre>
Create table manages	<pre>CREATE TABLE if not exists Manages (hotel_id INTEGER PRIMARY KEY, employee_ssn INTEGER NOT NULL UNIQUE CHECK (employee_ssn between 100000000 and 999999999), FOREIGN KEY (hotel_id) REFERENCES hotel (hotel_id) ON DELETE CASCADE, FOREIGN KEY (employee_ssn) REFERENCES employee (employee_ssn));</pre>

Create table employee_position

```
CREATE TABLE if not exists
employee_position (
    employee_ssn integer NOT NULL CHECK (
        employee_ssn between 100000000
and 999999999
    ),
    hotel_id INTEGER NOT NULL,
    position VARCHAR(30) NOT NULL CHECK (
        position IN (
            'Doorman',
            'Concierge',
            'Valet',
            'Security',
            'Front desk agent',
            'Housekeeper',
            'Maintenance',
            'Cleaner',
            'Server',
            'Bartender',
            'Chef'
        )
    ),
    PRIMARY KEY (
        employee_ssn,
        hotel_id,
        position
    ),
    FOREIGN KEY (hotel_id) REFERENCES
hotel (hotel_id) ON DELETE CASCADE,
    FOREIGN KEY (employee_ssn) REFERENCES
employee (employee_ssn) ON DELETE CASCADE
);
```

Create table Room

```
CREATE TABLE if not exists Room (
    hotel_id integer NOT NULL,
    room_number integer NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    capacity integer NOT NULL,
    view_type VARCHAR(20),
```

	<pre> extendable BOOLEAN NOT NULL, damages BOOLEAN NOT NULL, PRIMARY KEY (hotel_id, room_number), FOREIGN KEY (hotel_id) REFERENCES Hotel (hotel_id), CHECK (price > 0), CHECK (capacity > 0), CHECK (view_type IN ('Sea', 'Mountain')), CHECK (NOT(capacity = 1 AND extendable = TRUE))); </pre>
Create table amenity	<pre> create table if not exists amenity (hotel_id integer not null, room_number integer not null, amenity varchar(20), primary key (hotel_id, room_number, amenity), FOREIGN KEY (hotel_id, room_number) REFERENCES Room (hotel_id, room_number), CHECK (amenity IN ('TV', 'AC', 'WiFi', 'Fridge', 'Microwave', 'Iron')) </pre>

	<pre>)); </pre>
Create table customer	<pre> create table if not exists customer (first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, street_number integer not null check (street_number > 0), street_name varchar(50) not null, city varchar(50) not null, province varchar(50) not null, zip varchar(6) not null CHECK (zip ~ '^[A-Za-z][0-9][A-Za-z][0-9][A-Za-z][0-9] \$'), customer_id INTEGER primary key CHECK (customer_id between 100000000 and 999999999), registration_date date not null check (registration_date <= CURRENT_DATE)); </pre>
Create table renting_booking	<pre> create table if not exists renting_booking (hotel_id integer not null, room_number integer not null, customer_id INTEGER not null check (customer_id between 100000000 and 999999999), renting_booking_id integer not null, checkin_date date not null, checkout_date date not null, employee_responsable integer not null </pre>

	<pre> CHECK (employee_responsable between 100000000 and 999999999), booking boolean not null, booking_date date, PRIMARY KEY (renting_booking_id), FOREIGN KEY (hotel_id, room_number) REFERENCES Room (hotel_id, room_number), FOREIGN KEY (customer_id) REFERENCES Customer (customer_id), FOREIGN KEY (employee_responsable) REFERENCES Employee (employee_ssn), Check (checkin_date < checkout_date), CHECK ((booking = TRUE AND booking_date IS NOT NULL) OR (booking = FALSE)), UNIQUE (hotel_id, room_number, customer_id, checkin_date, checkout_date, employee_responsable, booking, booking_date)); </pre>
Create function check_checkin_not_past	<pre> CREATE OR REPLACE FUNCTION check_checkin_not_past() RETURNS TRIGGER AS \$\$ BEGIN IF NEW.checkin_date < CURRENT_DATE </pre>

	<pre> THEN RAISE EXCEPTION 'checkin_date (%) cannot be before today (%)', NEW.checkin_date, CURRENT_DATE; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; </pre>
Create trigger trg_check_checkin_not_past	<pre> CREATE TRIGGER trg_check_checkin_not_past BEFORE INSERT OR UPDATE ON renting_booking FOR EACH ROW EXECUTE FUNCTION check_checkin_not_past(); </pre>
Create function check_renting_overlap	<pre> CREATE OR REPLACE FUNCTION check_renting_overlap() RETURNS TRIGGER AS \$\$ BEGIN IF EXISTS (SELECT 1 FROM Renting_booking r WHERE r.hotel_id = NEW.hotel_id AND r.room_number = NEW.room_number AND r.renting_booking_id <> NEW.renting_booking_id AND NEW.checkin_date < r.checkout_date AND NEW.checkout_date > r.checkin_date) THEN RAISE EXCEPTION 'This room already has a renting during the selected date range.'; END IF; </pre>

	<pre> RETURN NEW; END; \$\$ LANGUAGE plpgsql; </pre>
Create trigger trg_check_renting_overlap	<pre> CREATE TRIGGER trg_check_renting_overlap BEFORE INSERT OR UPDATE ON Renting_booking FOR EACH ROW EXECUTE FUNCTION check_renting_overlap(); </pre>
Create table archived_renting_booking	<pre> create table if not exists archived_renting_booking (hotel_id integer not null, archived_renting_booking_id integer primary key, room_number integer not null, checkin_date date not null, checkout_date date not null, booking_date date, booking boolean not null, customer_id INTEGER not null check (customer_id between 100000000 and 999999999), employee_responsable integer not null CHECK (employee_responsable between 100000000 and 999999999), check (checkin_date < checkout_date), CHECK ((booking = TRUE AND booking_date IS NOT NULL) OR (booking = FALSE)), UNIQUE (hotel_id, </pre>

	<pre> room_number, checkin_date, checkout_date, employee_responsable, booking, booking_date, customer_id));</pre>
--	--

Section 2: Additional SQL Code and Application SQL

Preface:

As previously stated, the SQL queries used in the web application are heavily formatted to allow for flexibility, but as a result, this decreases readability. Thus, some SQL statements from the application have been recreated with static elements rather than variables to further showcase what is happening on the database side. It is important to note that not all SQL queries used in the application have been recreated; however, four examples that meet the requirements of the assignment (four queries including one aggregation use case and one nested use case) have been prepared. Further, all queries used in the application are fully functional, this recreation is for explanation and display purposes only.

Recreated queries and all database creation files can be found [here](#):
/src/main/resources/database

Queries used in the application can be found in any DAO class or [here](#):
/src/main/java/com/example/dao

Example 1: Change Booking to Renting

- File in database directory: example1.sql
- Explanation: Query looks for booking with ID=2 and updates it to renting status if it has booking status.
- Method in application: BookingDAO.convertToRenting. Found [here](#).

Example 2: Get Hotel Most Expensive Room (Nested Requirement)

- File in database directory: example2.sql
- Explanation: Query looks for most expensive room in a hotel to allow users to compare price points at the same hotel.
- Method in application: RoomDAO.getMostExpensiveRooms. Found [here](#).

Example 3: Get Number of Available Rooms in City (Aggregation Requirement)

- File in database directory: example3.sql
- Explanation: Query looks for the number of total available rooms across all hotel chains in a city for the time period requested. Uses Count() to get the number of rooms and to include aggregation.
- Method in application: RoomDAO.getAvailableRoomsInCity. Found [here](#).

Example 4: Search Scenario

- File in database directory: example4.sql
- Explanation: Query looks for available rooms with the following requirements:
 - Located in Ottawa
 - Five star rating
 - At least a capacity of two people
 - Price is at most \$300
 - Room is not booked yet for wanted time period
 - Hotel has at least five rooms total
 - Room is extendable
- Method in application: RoomDAO.searchRooms. Found [here](#).